

Scalable Strategies for GitHub Actions Workflow Run Cleanup (2023–2025)

Chunking Models for Bulk Run Processing

Cleaning up thousands of workflow runs is best done in **batches (chunks)** rather than all at once. An effective chunking strategy balances throughput and safety:

- **Batch Size Recommendations:** Use the GitHub API's pagination to retrieve runs in blocks (the API allows up to 100 runs per page) ¹. Many scripts default to ~20–30 runs per page unless specified, so explicitly setting `per_page=100` can reduce API calls ². In practice, batch sizes around **100 runs per chunk** are common, providing a manageable unit of work and ensuring each batch stays well within rate limits. Some workflows use even larger chunks (e.g. 500–1000 runs) to expedite deletion, but this requires careful rate-limit handling (discussed below). At the extreme, listing 1000 runs in one go is possible (the API will paginate under the hood up to 1000 results for certain queries) ³, but deleting 1000 runs back-to-back risks throttling if not throttled or delayed appropriately.
- **Trade-offs – Small vs Large Chunks:** Smaller chunks (e.g. 50–100 runs) are **safer** and allow easier progress tracking and checkpointing. They consume fewer API calls per batch (100 deletions = 500 API points) and give the script opportunities to pause or adjust between batches. Larger chunks (e.g. 500+ runs) can improve overall speed by reducing loop overhead, but they must be managed to avoid hitting secondary rate limits. For example, deleting 1000 runs without delays would generate 1000 API delete calls (each a write operation), far exceeding the safe threshold of ~180 write requests per minute ⁴. Thus, large batches should incorporate deliberate pacing or be split internally.
- **"96" as a Safe Batch Size:** A batch size of **96 runs** is sometimes cited as a sweet spot, because it provides a cushion under certain rate limits. For instance, 96 deletions correspond to 480 API points (since each delete is 5 points) ⁵, which is comfortably below the 900 points/minute secondary limit for REST API calls ⁴. In practical terms, ~96 deletions can be completed in about a minute without triggering the 900-point throttle, even if done in quick succession. This leaves headroom for other API calls (like archiving operations) in the same minute. While 100 deletions (500 points) are also generally fine, using 90–100 as a batch size with short pauses between batches ensures you stay below the enforcement trigger. The exact number need not be 96 specifically – the key is to batch on the order of a few dozen up to 100 and **avoid one giant loop** with no breaks.
- **Iterative Processing and Memory Use:** Processing in chunks also prevents excessive memory use or long single-API calls. You can iterate through pages of runs, handle each page (archive & delete those runs), then move to the next. GitHub CLI's `--paginate` option automates iterating through result pages ², which is useful for scripting. This approach was used in community scripts where, for example, only ~25 runs were deleted per iteration by default due to pagination, until `--`

`paginate` was added to continue through all pages ². The takeaway is to **chunk the workload**: fetch a set number of run IDs, process them, then proceed, which provides natural checkpoints and easier recovery if something goes wrong mid-way.

Rate Limiting Techniques and Throughput Control

GitHub's API rate limits are a crucial factor in scaling deletion of workflow runs. Best practices revolve around respecting both the primary hourly limits and the secondary short-term limits:

- **Primary Rate Limits (Hourly):** An authenticated user with a Personal Access Token (PAT) can make up to **5,000 API requests per hour** ⁶. However, if you use the built-in `GITHUB_TOKEN` in Actions, **the limit is 1,000 requests per hour per repository** ⁷. This is a "primary" rate limit. For large cleanups, prefer using a PAT (with `repo` and `workflow` scopes) to get the higher 5,000/hour quota. If your deletion job is running as a workflow in the same repo, be mindful that the default `github.token` will cap at 1000/hour ⁷. Hitting the primary limit will result in 403 errors until the hour resets, so plan chunk sizes and delays to stay within ~5000 deletions per hour (for PAT) or 1000/hour (for `GITHUB_TOKEN`). If tens of thousands of runs need removal, the process may need to span multiple hours or use multiple tokens in sequence (or you can request a temporary rate limit increase from GitHub support for a one-time cleanup).
- **Secondary Rate Limits (Concurrent & Per-Minute):** GitHub imposes additional limits on bursty activity to maintain stability ⁸. Notably, **no more than 100 API requests should be in flight concurrently**, and for REST calls, **no more than 900 points per minute** are allowed ⁴. Since most write operations (POST/DELETE) count as 5 points each ⁵, this translates to **roughly 180 write requests per minute** maximum to avoid triggering secondary throttling ⁴. In practice, staying a bit below that (e.g. ~150 deletes/minute) adds a safety margin. Concurrency should also be limited – even if you spawn many threads, ensure fewer than 100 API calls run at the exact same time ⁴. A common strategy is to introduce a **small delay (e.g. 0.5–1 second)** between deletion calls when running synchronously, or to run a limited pool of worker threads (for example, 5–10 concurrent deletions) to naturally throttle the rate. This prevents hitting the "secondary" cap where GitHub returns errors for too many rapid requests. It's also wise to **spread calls across endpoints** when possible – e.g. mix in some artifact or log downloads between deletions – because the 900/minute limit is per endpoint. If all calls are hitting the same delete endpoint, the 180/minute rule is the bottleneck ⁴.
- **Monitoring and Adapting to Rate Limits:** Integrate checks for the API's rate limit headers in your script. Every response includes `X-RateLimit-Remaining` and `X-RateLimit-Reset` headers ⁹. By monitoring `Remaining`, you can predict when you'll exhaust the hourly quota and back off accordingly. For example, if you notice the remaining count dropping near zero, the script can sleep until `X-RateLimit-Reset` (timestamp when the window resets) ⁹. For secondary limits, GitHub may return a 429 Too Many Requests or a special error if you exceed burst limits; if you catch those, implement an exponential backoff (wait a few seconds and try again, or slow the pace globally). **Adaptive throughput** can be achieved by dynamically adjusting delay or concurrency: start faster, but if responses begin to slow or any rate-limit warnings appear, throttle down. Conversely, if throughput is well below limits, you might cautiously increase parallelism. The goal is to **maximize speed without crossing into throttling**. Logging current request rates and successes can help tune

this. One best practice is to target ~80% of the known limit – for instance, aim for ~150 deletes/minute instead of 180 – to avoid edge-triggering a limit.

- **GraphQL and Alternative Approaches:** In some cases, the GitHub GraphQL API can batch multiple operations in one call, potentially reducing the number of HTTP requests. However, GraphQL mutations still count against rate limits (5 points per mutation), and the complexity of writing a multi-delete GraphQL query (and handling partial failures) can be high. The GraphQL secondary limit is 2,000 points/minute ¹⁰, which in theory could allow ~400 deletions/min (if done as separate mutations). But doing this in one large GraphQL call is not straightforward, and you might hit query complexity limits. In general, **using the REST API with careful pacing tends to be simpler and sufficiently efficient** for run cleanup. GraphQL could be considered if you need to, say, query for runs and delete them in one round trip, but most implementations stick to REST for deletions. Another tactic to reduce API calls is to **increase the amount of work done per call**: for example, use one API call to list 100 runs (max page size) and then loop through those IDs in memory to delete with subsequent calls. This at least amortizes the listing overhead. Also prefer filtering server-side when listing (the list endpoint supports filters by branch, status, event, etc.) to avoid retrieving runs you won't delete. But note that listing with certain filters might cap at 1000 results ³, so if you have more than 1000 runs to delete, you may need to chunk by date or do multiple queries (e.g. filter older than X date, then older than Y date, etc.). Being mindful of these nuances keeps you within rate limits while cleaning up efficiently.

Automation Strategies for Archiving and Deleting Runs

Automating the archive-and-delete process ensures consistency and saves time. Below are strategies and tools for implementing the cleanup:

- **Pipeline: Archive → Delete Workflow:** Set up a two-stage pipeline for each run – first **archive the run's data**, then **delete the run** from GitHub. Archiving typically means downloading the run's logs and artifacts to external storage. You can use GitHub's API to download logs (there's an endpoint that provides a ZIP of all logs for a run) ¹¹ and to list/download artifacts ¹². The GitHub CLI simplifies this: for example, `gh run download <run-id>` will pull all artifacts of a run ¹³ (and you can retrieve logs via `gh run view <run-id> --log` or the API). By scripting the pipeline, you ensure **no run is deleted before its data is safely archived**. This can be done with a queue system: one thread (or step) fetches and saves data, then signals another to delete that run. In practice, you might archive runs in batches (e.g. download logs for 10 runs in parallel), and once confirmed, proceed to delete those 10, then move on. This overlap hides latency (download time) and keeps the pipeline busy, improving overall throughput.
- **Parallelism and Asymmetric Pipeline:** It's often beneficial to run archiving and deletion in parallel, but with **different degrees of concurrency** for each. Archival operations (downloading logs/artifacts) are I/O-bound and **do not count heavily against API rate limits** after the initial request. For example, the API call to get a log download URL is 1 request, while the actual download from cloud storage doesn't count toward GitHub API limits. Thus, you could run several archive downloads concurrently (limited mainly by your network bandwidth and the size of artifacts). Deletions, on the other hand, are API-bound and subject to strict rate limits, so you might only run a few deletion threads at a time. For instance, you could configure 5 threads fetching logs/artifacts while 1–2 threads handle deletions. This **asymmetric design** keeps the slower, API-limited stage (deletion)

from gating the entire process – the archive threads can keep preparing the next runs to delete. It also prevents a pile-up of API calls: even if archives produce data quickly, deletion threads ensure you only send out delete requests at a controlled rate ⁴. Tuning the ratio is an optimization task (covered in the next section), but the principle is to separate concerns: maximize archive parallelism until system/IO limits, but throttle deletions to comply with API limits.

- **Using GitHub CLI and Scripts:** The **GitHub CLI** (`gh`) is a powerful tool for this task. It provides direct access to the GitHub REST API and can be used in shell scripts. For example, a one-liner using `gh` can list and delete runs:

```
gh run list --limit 100 --json databaseId -q '.[].databaseId' \
| xargs -n1 -I {} gh api "repos/:owner/:repo/actions/runs/{" -X DELETE
```

This command uses `gh run list` to fetch run IDs (with a `--limit` or `--paginate` to go beyond the default 20) and then `xargs` to delete each run via the API ¹⁴. A similar example from Stack Overflow shows filtering out runs on the default branch and deleting the rest in bulk ¹⁵. The GitHub CLI handles pagination if you add `--paginate`, so it can retrieve all run IDs even for thousands of runs, which you can then pipe into deletion. These CLI approaches are quick to set up and useful for one-off cleanups or cron jobs.

- **Open-Source Actions and Tools:** In the community, specialized Actions and scripts have been created for run cleanup. One notable example is **Matraks/delete-workflow-runs** (v2 on the Marketplace), which is a GitHub Action that deletes older runs based on specified criteria. It allows you to set a retention period (e.g. 30 days) and a minimum number of runs to keep per workflow (e.g. keep last 6 runs) ¹⁶. Using such an Action in a scheduled workflow (e.g. running monthly or weekly) is a **best practice to automate ongoing cleanup** ¹⁷. The action will ensure that for each workflow, any runs older than X days beyond the newest N runs are deleted. Similarly, it provides filters (by workflow name, run conclusion, branch, etc.) and a **dry_run mode for safety** ¹⁸. By configuring parameters like `keep_minimum_runs: 6` (default) ¹⁶ or changing it to 100, you can approximate the goal of “keep ~100 visible runs” per workflow. Another open-source tool, for example, is a Python script by the community that deletes runs but always keeps the latest run on each branch ¹⁹ ²⁰ – this strategy ensures at least one recent run is preserved per branch. These tools abstract a lot of the API handling and offer convenience features like **progress output** and filtering.

- **Real-time Progress and Visibility:** When running a long cleanup (which might take minutes or hours for thousands of runs), provide real-time feedback. A simple approach is periodic logging: e.g., print a message every 50 runs deleted, or display the current batch number. More advanced, use a **progress bar or spinner** in CLI tools – libraries like `tqdm` (for Python) or even just updating a single-line output with the percentage complete can greatly enhance visibility. Real-time timers and counters can show, for example, “Archived and deleted 500 of 5,000 runs – 10% complete, ~5 minutes elapsed, ~45 minutes remaining (ETA).” This is important for operator confidence, allowing you to spot if the process hangs or slows down. If using the GitHub CLI or actions, you won’t have an interactive progress bar, but you can still output timestamps and counts to the log so you can monitor it in the Actions console. For instance, echo a line like “Deleting run 12345... (200/1000)” as you go. In a GUI or script scenario, a progress bar updated every few seconds gives immediate

insight. **Timers** can be implemented by capturing the start time and occasionally calculating elapsed time and extrapolating based on how many runs are done vs remaining. Including an ETA that recalculates over time (especially if throughput changes due to rate limiting) provides a dynamic view of completion. All these measures ensure the process isn't a "black box" – you have ongoing visibility and can abort if something seems off.

- **Safety Checks (Dry Runs and Confirmation):** Automation should include safeguards to prevent catastrophic mistakes. One safeguard is a **dry-run mode**, where the script goes through the motions (listing runs, deciding which would be deleted) and logs those, but **does not actually delete anything**. The aforementioned Marketplace action supports `dry_run: true` to list targeted runs without deletion ¹⁸. In a custom script, you can implement a `--dry-run` flag that simply prints the run IDs that would be removed or saves them to a file. Review this output to ensure the logic is correct (e.g. it's not trying to delete the last 100 instead of the older ones!). Another safety mechanism is requiring manual confirmation before mass deletion. For example, your script could prompt "Are you sure you want to delete 4,900 workflow runs? (yes/no)" before proceeding. This gives a human a chance to verify parameters. Also consider **scoping** your deletion carefully: target a specific workflow or branch if appropriate. If your goal is specifically to keep 100 total runs, make sure the script is indeed selecting the oldest runs across the repository, not accidentally wiping out recent ones. A mistake in a filter or ID list could lead to deleting the wrong runs, so test on a small scale first (e.g. try deleting just 1–2 runs via script to verify it works as expected). Automation is powerful, but with great power comes the need for careful validation.

Optimization Heuristics and Best Practices

Once the basic pipeline is in place, further optimizations and reliability improvements can be applied. These heuristics help maximize speed while ensuring the process can be paused, resumed, and executed safely:

- **Asymmetric Parallelism & Pipeline Tuning:** As mentioned, treating archiving and deletion as separate stages allows tuning their throughput independently. A heuristic is to **measure each stage's throughput** (e.g. how many runs archived per minute vs deleted per minute) and scale resources accordingly. If archiving logs/artifacts is slower (for instance, downloading large artifacts could be the bottleneck), you might increase the number of archive threads until your network or disk throughput is saturated, ensuring you're always preparing the next items for deletion. Conversely, if deletion is the bottleneck due to rate limits (e.g. capped at ~3 deletes/sec), piling on more deletion threads won't help – they'll just end up idle or cause throttling. Instead, you could devote extra threads to archiving or even to other tasks (like verifying archives). This **asymmetric design** means, for example, you might be archiving 10 runs in parallel while only deleting 2 at a time. The result is the archival stage produces a queue of archived runs ready to delete, and the deletion stage steadily works through them at the maximal allowed rate. This keeps all parts of the pipeline busy without overloading the API. Monitor the pipeline: if the deletion queue is often empty waiting for archives, archive more in parallel; if the queue backs up too much, you might archive a bit slower to not get too far ahead (or it's fine if storage is not an issue – backlog of archived-but-not-deleted is safe as long as you track them).
- **Adaptive Throughput Calibration:** Conditions can change during execution – network speeds fluctuate, API responses might slow down if GitHub's under load, etc. Build in **feedback mechanisms** to adapt throughput. For example, measure the time taken for the last N deletions and

adjust sleep intervals accordingly. If each deletion API call is returning very fast (say 0.1s each), you might be able to slightly increase concurrency or reduce delay until you approach the known safe limit. If responses start to lag or you approach 429 errors, automatically back off the rate. One adaptive strategy is **token bucket rate limiting**: allow a certain number of operations per second and refill the allowance gradually, which smooths bursts. Another is to use GitHub's secondary limit info: remember that ~180 writes/min is the hard ceiling ⁴. If you set your deletion loop to aim for ~150/min and monitor success, you can tune it. For instance, start at 2/sec and if after a minute you see no issues, try 3/sec; if a throttling warning appears, drop to 2/sec again. This kind of adaptive control can significantly shorten the total time by using as much throughput as allowed without crossing into failure. Also, keep an **eye on the remaining hourly quota** – if you see you'll run out before finishing, the script could proactively pause when near the limit rather than failing mid-transaction. It could wait until the top of the next hour when the rate-limit resets (using the `X-RateLimit-Reset` timestamp) ⁹, then resume. Adaptive algorithms make the cleanup robust against variability and ensure optimal speed under the current constraints.

- **ETA Modeling and Dynamic Updates:** Estimating the completion time helps with planning and monitoring. A simple heuristic for ETA is:
$$\text{ETA} = (\text{TotalRuns} - \text{ProcessedRuns}) / \text{CurrentThroughput}$$
. Since throughput might change (especially if you throttle down after a burst), recalculate ETA periodically. For example, after each batch of 100 deletions, recompute the average deletions per minute so far and update the remaining time estimate. Communicate these updates in your progress output. This not only manages expectations but also helps detect if throughput has unexpectedly dropped (if an ETA suddenly balloons, something might be wrong or rate limits kicked in). Modern CLI UIs can even show moving average speeds. The goal is **not just to guess once, but to adjust the estimate** as conditions evolve. Over a long run, the ETA might start at 2 hours, then settle to 1.5 hours if the script ramps up speed, then maybe increase to 1.7 hours if a rate cap slows it. By modeling this, you can also decide whether to increase parallelism or accept the timeline. It effectively ties into adaptive throughput: if ETA is much higher than acceptable and you're not near limits, that's a signal you can try to speed up.
- **Deterministic Checkpointing and Resume Logic:** It's critical to design the process such that it can be stopped and resumed without confusion or duplication. A **deterministic checkpointing** approach means you have a clear record of what has been done at each step. One simple method is to sort runs by creation time (oldest first deletion, since we want to keep newest 100). If you always delete from oldest to newest, you can record the timestamp or run ID of the last deleted run. If the script crashes or is interrupted, you can restart it to pick up from the next run after the last recorded ID/time. This ensures idempotence – you won't accidentally delete something twice or skip any. For example, if you intend to delete all runs older than **Run #101** (the 101st newest), determine that cutoff ID/time at the start. Then your process knows exactly which runs to target (older than that). If paused, that cutoff remains the boundary. This is deterministic because no matter when you resume, the set of runs to delete doesn't change (aside from already-completed ones being gone). Alternatively, maintain a log file or database of deleted run IDs. On resume, the script can fetch the latest run list and subtract any IDs already in the "deleted" log to avoid repeats. It can also skip archiving for those already archived. Having this state means even if you run the cleanup in multiple sessions (say 2 hours today, 2 hours tomorrow), you won't lose your place.
- **Robustness against Failures:** Implement retry logic for transient failures. If a deletion fails due to a network glitch or a secondary rate limit, don't immediately give up. The script could catch the

exception, wait a few seconds, and retry that deletion. If it consistently fails for a particular run (perhaps due to an API issue), log the run ID and continue with others – you can review those problem cases later. Ensure that an error in one thread doesn't crash the whole program: use try/catch around API calls and perhaps a queue system that can continue even if one item had an issue. Checkpointing plays a role here too: if something fatal does happen, you have the progress saved and can restart without starting over.

- **Safety and Verification Measures:** Avoiding accidental data loss is paramount. A best practice is to **verify archives before deletion**. This could mean checking that the log file was downloaded and has content, or that artifact files are not corrupted (maybe verify file sizes or checksums if available). Only after verification should the script mark that run as archived and proceed to delete. One could even implement a small delay between archive and delete of a single run to double-check (though if done synchronously, this slows things down; better to verify asynchronously and use a flag). Another safety heuristic is **keeping a minimum number of runs regardless of conditions** – for example, even if you intend ~100, maybe code the script to never delete runs that are among the latest 5 or 10 on the default branch as an extra cushion. This idea is similar to the `keep_minimum_runs` of 6 in the Marketplace action ¹⁶, which guarantees a floor. You could set your own floor (maybe 100 in this case) to be safe. Also, consider **excluding recent runs that might still be needed**: GitHub does not allow deleting a run that is in progress or very recent (the UI only allows deletion of completed runs, and any run <14 days old must be completed) ²¹. Ensure your script filters out non-completed runs (e.g., ignore runs with status `in_progress` or `queued`).
- **Post-Deletion Audit:** After the script finishes, cross-verify that only the expected runs were removed. You can list the remaining runs (there should be ~100, give or take new ones that ran during the process) and maybe compare their IDs with the backup of what was deleted. If you archived data, ensure the count of archived runs matches the count of deletions. It's a good idea to spot-check a couple of the newest remaining runs to confirm they are indeed the ones you intended to keep. If using a dry-run first, compare its output to the actual outcome to ensure all intended deletions happened and no extra.
- **Regular Maintenance vs One-time Cleanup:** Once you've reduced the visible runs to ~100, you'll likely want to **maintain** that state. A heuristic here is to automate a periodic cleanup (e.g. run the deletion script weekly or monthly) rather than letting thousands accumulate again ¹⁷. This way each cleanup cycle is smaller and faster, and you're always staying near the target of 100 recent runs. This can be done with a scheduled GitHub Action (using the cleanup action or your script on a cron schedule) or an external cron job. The initial deep-clean is the hardest; subsequent incremental cleanups can just remove, say, runs older than 90 days or beyond the latest 100, whichever criterion kicks in first.

By following these optimization and safety heuristics, you create a resilient cleanup process. It will be **scalable** (handling thousands of runs in chunks without blowing up), **rate-limit-aware** (staying within GitHub's rules by design, using adaptive throttling ⁴ ⁶), and **fault-tolerant** (able to resume and unlikely to mistakenly delete important data). Combining community best practices from 2023–2025 – from chunked deletions, to parallel archive-delete pipelines, to careful rate management – will enable you to reduce your GitHub Actions workflow runs to a tidy ~100 visible entries while safely archiving the rest for posterity.

Sources:

- GitHub API Rate Limit Documentation ⁶ ⁴ (primary and secondary limits)
- GitHub Actions Workflow Runs API Docs ¹ ³ (pagination and result limits)
- Stack Overflow and Community Scripts on Bulk Deletion ² ¹⁵
- Open-Source GitHub Action *delete-workflow-runs* (v2) ¹⁶ ¹⁷ (retention settings and scheduling tips)
- *delete-workflow-runs* Action Documentation ¹⁸ (dry-run safety feature)
- GitHub CLI Official Examples ¹⁴ (bulk deleting runs with `gh` CLI)
- GitHub REST API Guide ⁹ (monitoring rate-limit headers for adaptive control)

¹ ³ ¹¹ REST API endpoints for workflow runs - GitHub Docs

<https://docs.github.com/en/rest/actions/workflow-runs>

² GitHub - scottgriv/bash-github_delete_workflow_runs: Mass delete all GitHub Actions workflow runs for a specific repository using GitHub CLI.

https://github.com/scottgriv/bash-github_delete_workflow_runs

⁴ ⁶ ⁷ Rate Limit Cheatsheet for Self-Hosting Github Runners - WarpBuild Blog

<https://www.warpbuild.com/blog/rate-limits-self-hosted-runners>

⁵ ⁸ ⁹ ¹⁰ Rate limits for the REST API - GitHub Docs

<https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api>

¹² REST API endpoints for GitHub Actions artifacts

<https://docs.github.com/en/rest/actions/artifacts>

¹³ gh run download - GitHub CLI

https://cli.github.com/manual/gh_run_download

¹⁴ Delete GitHub workflow runs using the gh cli :: blog.oddbit.com

<https://blog.oddbit.com/post/2022-09-22-delete-workflow-runs/>

¹⁵ Delete a workflow from GitHub Actions - Stack Overflow

<https://stackoverflow.com/questions/57927115/delete-a-workflow-from-github-actions>

¹⁶ ¹⁷ ¹⁸ GitHub - 1uphealth/delete-workflow-runs-multi-repo-action: An action to delete workflow runs in an array of repositories.

<https://github.com/1uphealth/delete-workflow-runs-multi-repo-action>

¹⁹ ²⁰ GitHub - grindsa/gh_workflow_runs_clean: Script to cleanup github workflow runs and artifacts

https://github.com/grindsa/gh_workflow_runs_clean

²¹ Deleting a workflow run - GitHub Docs

<https://docs.github.com/actions/managing-workflow-runs/deleting-a-workflow-run>